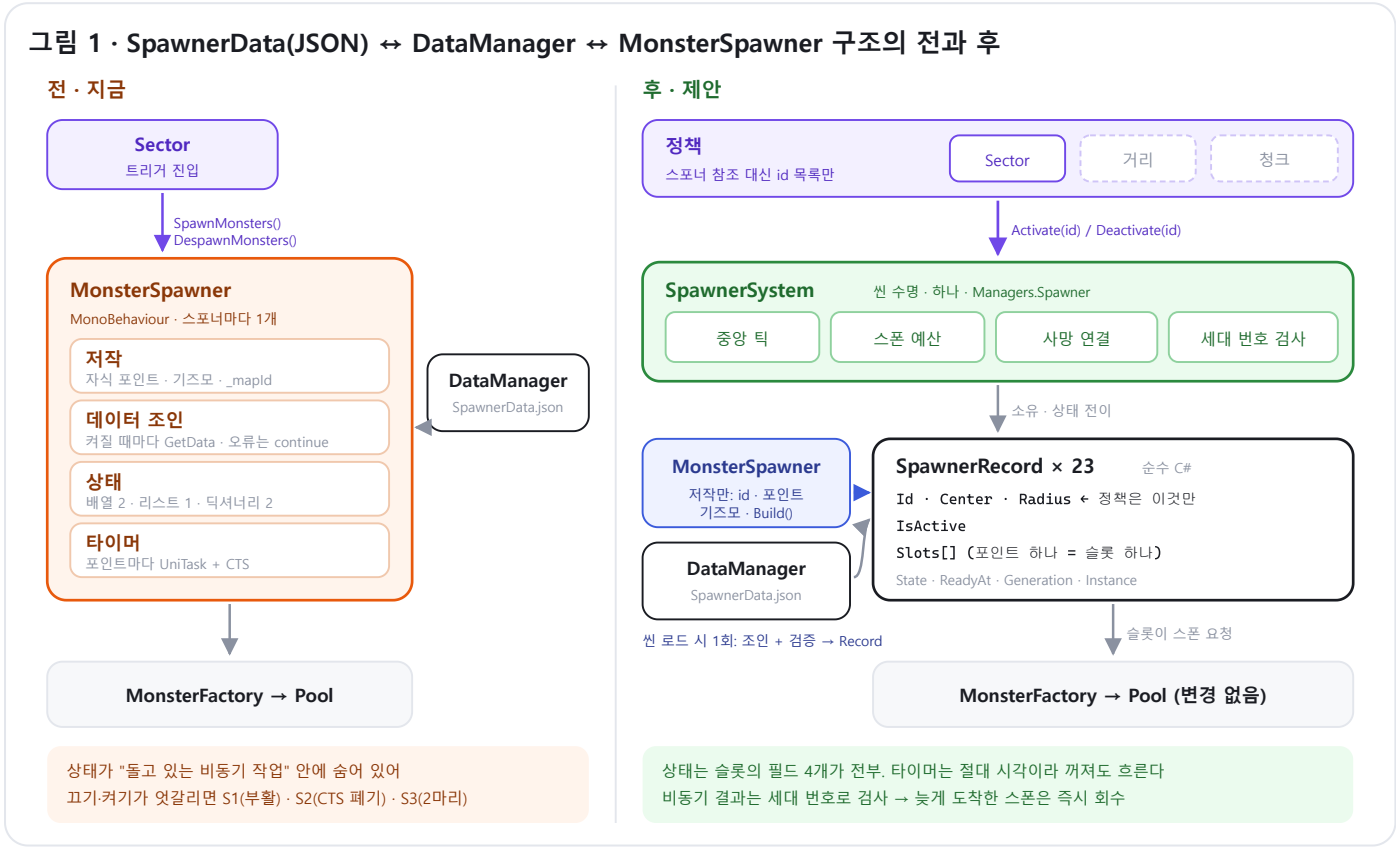


한 클래스의 네 가지 일을 네 층으로 나눈다

방법 1(손 배치 + ID 표)을 채택했으므로 데이터 형식은 그대로 두고 런타임만 다시 짤다. `MonsterSpawner` 가 떠 안던 저작 · 데이터 조인 · 상태 · 타이머를 층별로 떼어 내고, 포인트 하나의 상태를 명시적인 상태 기계로 만든다. 그러면 포인트마다 돌던 비동기 작업이 사라지고 S1~S3이 구조적으로 없어진다.

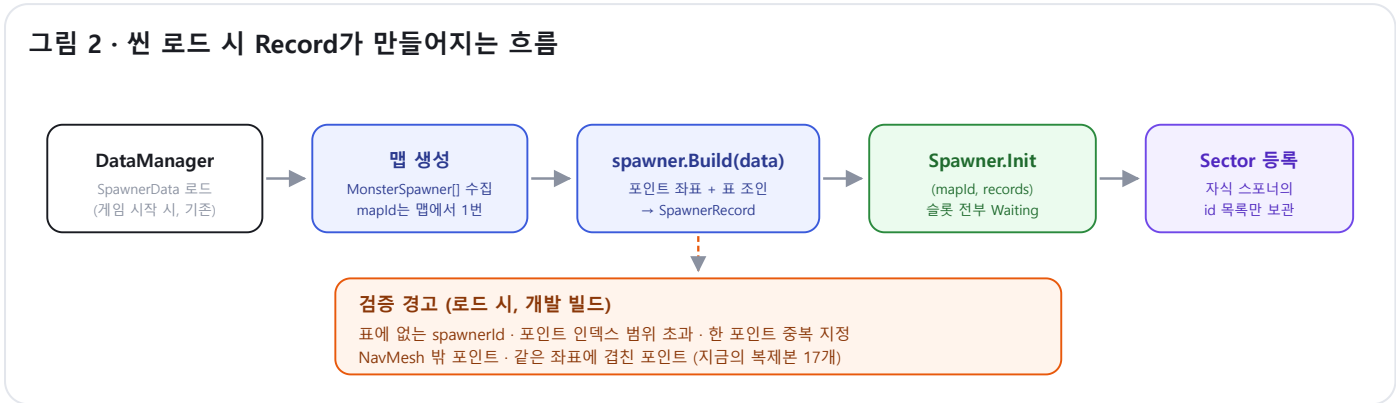
1. 전 / 후

왼쪽이 지금, 오른쪽이 제안. 아래층(MonsterFactory · Pool · 몬스터 생애)은 Phase 1 결과 그대로



2. 씬 로드 — 한 번만 조인하고 검증한다

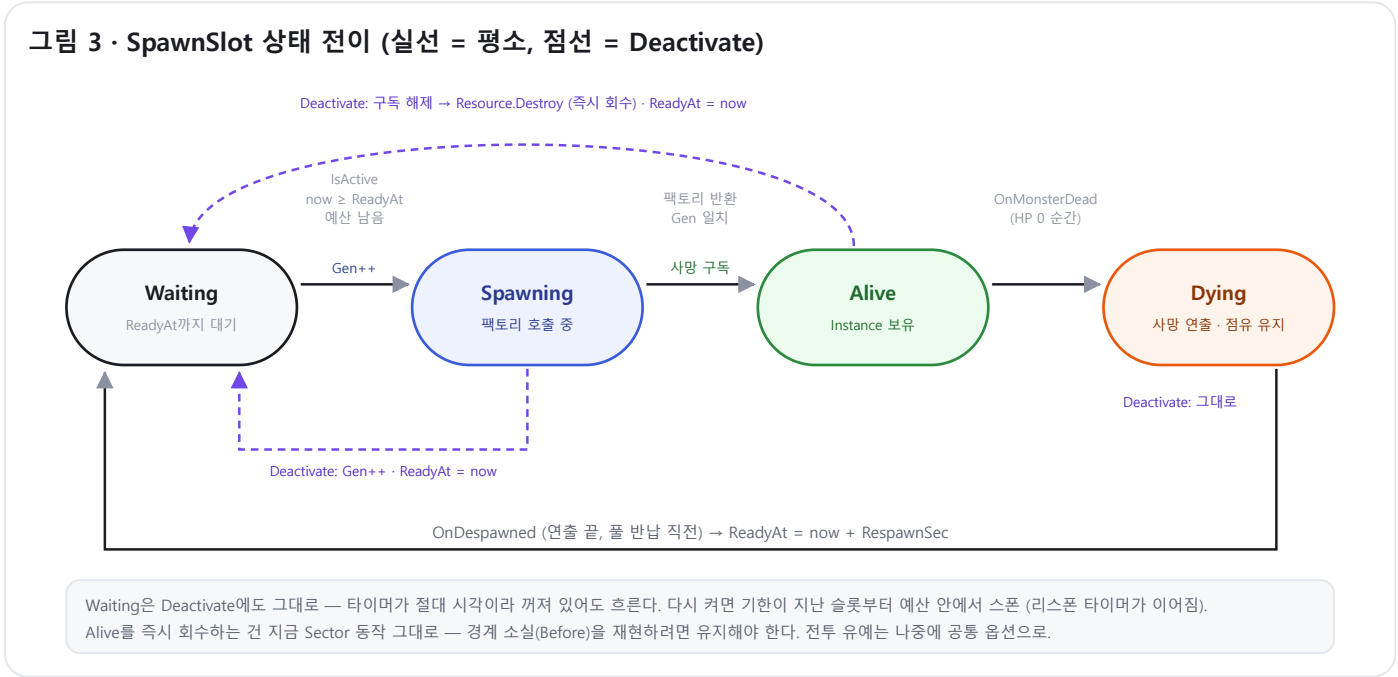
지금은 "켜질 때마다" 조인해서 오류가 늦게, 조용히 드러난다



첫 스폰 대기(아래 5절 결정)에 따라 Init 에서 슬롯의 ReadyAt 을 0(켜지는 즉시)으로 돌지, 활성화 시각 + initialDelay 로 돌지가 갈린다.

3. 슬롯 상태 기계 — 이 문서의 핵심

포인트 하나 = 슬롯 하나. 상태는 4개, 전이는 누가 일으키는지가 정해져 있다



전이	일으키는 것	시스템이 하는 일
Waiting → Spawning	OnUpdate 틱	조건 3개 확인 → Generation++ → 팩토리 호출 (씬 CTS 하나 전달)
Spawning → Alive	팩토리 반환	반환 시 받아 둔 번호와 슬롯의 Generation 이 같을 때만 채택. 다르면 Resource.Destroy (늦게 도착한 결과)
Alive → Dying	MonsterStat.OnMonsterDead	슬롯 상태만 바꿈. 점유는 유지 → 이 동안 같은 포인트에 새 몸 없음 (S3)
Dying → Waiting	OnDespawned	딕셔너리에서 연결 제거, ReadyAt = now + RespawnSec
Spawning → Waiting	Deactivate	Generation++ 만으로 진행 중인 요청을 무효화. CTS를 포인트마다 둘 필요 없음 (S2)
Alive → Waiting	Deactivate	연결을 먼저 끊고 Resource.Destroy (→ Pool.Push → OnDespawn). 사망 경로가 아니므로 OnDespawned는 안 불림

4. 세 가지 장면을 따라가 보기

가장 자주 일어나는 것 하나, 지금 버그가 나던 것 둘



5. 불변식 — 매 프레임 참이어야 하는 것

개발 빌드에서 검사해 두면, 거리 정책을 붙일 때 "몬스터 층이 깨끗하다"는 증거가 된다

#	불변식	막는 것
I1	꺼진 레코드의 슬롯은 Alive · Spawning이 아니다 (Dying 은 허용)	S1 꺼진 구역 부활, 유령 스폰
I2	슬롯 하나에 인스턴스는 최대 1개, 인스턴스 하나는 슬롯 최대 1개	S3 한 포인트 2마리
I3	사망 연결 디렉터리 수 = Alive + Dying 슬롯 수	구독 누수, 연결 잔존
I4	HP바 수 = 활성 몬스터 수 (Phase 1 조건 재사용)	HP바 잔존
I5	풀 즉석 증설 = 0 ([PoolGrow])	스폰 비용이 정책마다 달라지는 것 (V2 · V3)

6. 코드 뼈대

이름-시그니처는 제안. 실제 구현에서 바뀔 수 있다

```
// — 런타임 데이터 - MonoBehaviour 아님. 합성 N개도 같은 형태 (청크 스케일 테스트)
public enum SlotState { Waiting, Spawning, Alive, Dying }

public sealed class SpawnSlot
{
    public readonly int MonsterId; public readonly float RespawnSec;
    public readonly Vector3 Position; public readonly Quaternion Rotation;
    public SlotState State; public float ReadyAt; public int Generation;
    public BaseMonsterController Instance;
}

public sealed class SpawnerRecord
{
    public readonly int Id; public readonly Vector3 Center; public readonly float Radius; // 정책이 읽는 전부
    public readonly SpawnSlot[] Slots;
    public bool IsActive;
}

// — 저작 - 로직 없음. 씬 로드 때 한 번 Record로 바뀌고 끝
public class MonsterSpawner : MonoBehaviour
{
    [SerializeField] int _spawnerId; // _mapId 삭제 → 맵에서 한 번
    public int SpawnerId ⇒ _spawnerId;
    public SpawnerRecord Build(SpawnerData data); // 자식 좌표 + 표 조인 + 검증 경고
    void OnDrawGizmos(); // 기존 그대로
}

// — 시스템 - 씬 수명. Managers.Update에서 OnUpdate, 씬 전환에서 Clear
public class SpawnerSystem
{
    readonly Dictionary<int, SpawnerRecord> _records;
    readonly Dictionary<BaseMonsterController, SpawnSlot> _owner; // 사망 연결 (람다 구독 대신)
    CancellationTokenSource _sceneCts; // 씬에 하나
    public int SpawnBudgetPerFrame = int.MaxValue; // 공통 스위치 (기본 무제한)

    public void Init(int mapId, IEnumerable<SpawnerRecord> records);
    public void Activate(int spawnerId);
    public void Deactivate(int spawnerId); // 그림 3의 점선 전이
    public void OnUpdate(); // 합성 레코드의 기한 된 Waiting → 예산 안에서 스폰
    public void Clear();

    async UniTaskVoid SpawnAsync(SpawnSlot slot, int gen); // 반환 시 gen ≠ slot.Generation 이면 회수
    void HandleDead(BaseMonsterController m); // Alive → Dying
    void HandleDespawned(BaseMonsterController m); // Dying → Waiting
}

// — 정책 (지금은 Sector) - 스폰너 참조 대신 id만
public class Sector : MonoBehaviour
{
    int[] _spawnerIds; // Start에서 자식 MonsterSpawner의 id 수집
    public void ActivateSector() { foreach (var id in _spawnerIds) Managers.Spawner.Activate(id); }
    public void DeactivateSector() { foreach (var id in _spawnerIds) Managers.Spawner.Deactivate(id); }
}
```

몬스터 쪽 작은 변경 하나

BaseMonsterController.OnDespawned 를 Action → Action<BaseMonsterController> 로 바꾼다. 그래야 시스템이 핸들러 하나로 "누가 끝났는지" 알 수 있고, 생애마다 램다를 만들지 않는다. 두 이벤트(OnMonsterDead , OnDespawned)는 이 미 때 스폰에 초기화되므로(SetStat · OnSpawn) 팩토리 반환 뒤 구독해도 안전하다.

7. 파일 변화와 순서

구분	파일	내용
신규	SpawnerSystem.cs · SpawnerRecord.cs	시스템 · Record · Slot · SlotState
대폭 축소	MonsterSpawner.cs	210줄 → id · Build · 기즈모만
수정	Sector.cs · SectorManager.cs	스포너 참조 → id 목록
수정	Managers.cs · GameScene.cs	Managers.Spawner 등록 · OnUpdate · Clear / 맵 생성 뒤 Init
수정	BaseMonsterController.cs	OnDespawned 시그니처
정리	BackStreetMap.prefab	복제본 자식 17개 × 19개 스폰너 삭제

순서	작업	끝났다는 기준
1	Record · Slot + MonsterSpawner.Build	로드 시 23개 Record 생성, 검증 경고가 복제본을 잡음
2	SpawnerSystem + Sector 연결	지금과 같은 체감으로 스폰·사망·리스폰
3	불변식 I1~I5 검사 (개발 빌드)	Sector 10회 왕복 · 사망 직후 왕복에서 위반 0
4	프리팸 복제본 정리	검증 경고 0

정할 것 — 첫 스폰 대기
지금 JSON의 delay (363개 전부 5.0초)는 첫 스폰 대기과 리스폰 시간을 겸한다. 구역에 들어가면 5초 동안 몸이 없다. 두 정책에 똑같이 걸리지만 "들어갔는데 몸이 없다"는 체감 지표를 흐린다. **제안:** delay → respawnSec 로 의미를 고정하고, 첫 활성화는 즉시 스폰 (ReadyAt = 0). 필요하면 initialDelay (기본 0)를 추가.